

Dabout

Mattéo

Oral Projet Big Data

Partie Business :

Bonjour à tous. Si je vous dis "marchés financiers", "trading" ou "loi de l'offre et de la demande", vous imaginez probablement des traders en costume à Wall Street, des écrans remplis de graphiques incompréhensibles, ou des algorithmes de haute fréquence complexes.

Pourtant, ma toute première véritable expérience avec le monde de la finance, je ne l'ai pas eue en lisant le Financial Times ou en regardant un cours d'économie. Je l'ai eue dans ma chambre, adolescent, en jouant à un jeu vidéo. Plus précisément, un MMORPG : un jeu de rôle en ligne massivement multijoueur.

Dans ces jeux, il y a ce qu'on appelle "l'Hôtel des Ventes". C'est un endroit virtuel où des milliers de joueurs se retrouvent pour acheter et vendre des ressources : des épées, du minerai, du bois. Et ce qui est fascinant, c'est que les prix ne sont pas fixés par les développeurs du jeu. Ils sont dictés à 100 % par les joueurs. S'il y a une pénurie de bois sur le serveur, le prix explose. Si tout le monde se met à miner du fer, le marché est inondé et le prix s'effondre.

C'était passionnant. Sans m'en rendre compte, j'étais en train de faire du trading. J'achetais bas la nuit quand la demande était faible, et je revendais cher le week-end quand tout le monde était connecté.

C'est de là qu'est née l'idée de mon projet. Aujourd'hui, la finance et l'économie peuvent sembler effrayantes ou opaques pour les jeunes générations. Je suis convaincu qu'une première approche par la gamification, par le jeu vidéo, est la méthode la plus puissante pour démystifier ce milieu. Une vision pure et simple de l'économie : une ressource, un carnet d'ordres, et des prix qui fluctuent en temps réel, sous nos yeux.

C'est exactement ce que j'ai développé pour ce projet : un "MMORPG Market Simulator". Un simulateur de salle des marchés gamifié.

L'expérience utilisateur est simple. Vous commencez avec un capital de départ de 1000 dollars, et quelques ressources en stock : du bois, du fer, de l'or. Votre but ? Faire du profit. Acheter bas, vendre haut. Autour de vous, le marché vit. Des bots virtuels, simulant d'autres joueurs, génèrent en permanence des ordres d'achat et de vente, créant ainsi un marché chaotique, vivant, extrêmement volatile.

Sur le papier, c'est un jeu amusant. Mais sous le capot... c'est un véritable cauchemar d'ingénierie.

Pourquoi ? Parce que simuler un marché financier réaliste nous propulse de plein fouet dans les problématiques fondamentales du Big Data, notamment ce qu'on appelle la Vitesse et le Volume.

Imaginez : à chaque seconde, des dizaines, voire des centaines de "joueurs" cliquent sur le bouton "Acheter" ou "Vendre". Si on tente de gérer cette tempête de données, ce volume massif de microtransactions en temps réel, avec une architecture web et une base de données relationnelle traditionnelles, le serveur s'effondre. L'écran fige. Le joueur perd de l'argent à cause du lag, et l'expérience est ruinée.

Le véritable enjeu de mon business, ce n'est donc pas seulement d'afficher de beaux graphiques. C'est de concevoir une architecture capable d'encaisser une charge extrême, de traiter des événements en asynchrone, et d'afficher un prix en temps réel sans jamais ralentir.

Partie Problèmes et Solutions :

Pour comprendre comment résoudre ce défi technique, il faut d'abord comprendre pourquoi une architecture classique échoue dans ce cas précis.

Imaginons que j'utilise une base de données relationnelle standard, de type SQL, pour gérer mon marché. Dans un système SQL classique, pour garantir qu'il n'y a pas d'erreurs financières, la base verrouille la donnée le temps de l'opération. Si 100 joueurs cliquent sur "Acheter du bois" exactement à la même milliseconde, la base de données relationnelle traditionnelle va s'étouffer car elle gère très mal ce qu'on appelle une explosion de petites écritures simultanées. Elle va les traiter une par une, tout bloquer, et l'interface de tous les joueurs va complètement figer. C'est exactement le problème auquel a fait face Facebook avec les millions de "Likes" générés chaque seconde.

La première grande décision technique a donc été d'utiliser la programmation asynchrone pour séparer complètement les écritures des lectures.

Au lieu d'attaquer directement la base de données quand un joueur clique sur "Acheter", j'ai mis en place une file d'attente, une "Queue" asynchrone. Quand une commande est passée, elle est simplement déposée dans cette file d'attente. C'est quasi instantané pour le joueur, son interface ne bloque pas. De l'autre côté, j'ai codé un moteur de marché qui tourne en boucle en arrière-plan. Il récupère les ordres dans la file un par un et fait les calculs mathématiques à son propre rythme.

Pour tester la solidité de ce système et simuler la réalité d'un jeu vidéo massivement multijoueur, j'ai créé des bots. Ces bots sont de petits programmes autonomes qui tournent en continu et génèrent des ordres d'achat et de vente aléatoires. Ils inondent la file d'attente de requêtes. Cela crée volontairement ce qu'on appelle un environnement de "Heavy Writes", une tempête d'écritures, exactement comme Uber doit absorber des millions de positions GPS par seconde.

Maintenant que nous savons encaisser les ordres de ces bots et des joueurs sans planter, il reste un deuxième problème : comment afficher le prix du marché en temps réel à l'écran, sans aucun délai ?

C'est là qu'intervient la différence majeure entre la mémoire RAM et une requête SQL. Faire une requête SQL pour demander "Quel est le prix actuel du bois ?" demande d'aller lire sur le disque dur du serveur, de traiter la demande, puis de la renvoyer. C'est un processus lourd. La mémoire vive, la RAM, c'est l'inverse : c'est la mémoire à court terme de l'ordinateur. La lecture y est instantanée.

J'ai donc décidé de stocker les prix actuels du marché non pas dans une base de données sur disque, mais directement dans la mémoire vive de mon application, à la manière d'une base "Key-Value". Ainsi, quand l'interface du joueur a besoin du prix, elle interroge la RAM et la réponse est immédiate.

Cette réflexion m'a poussé à structurer l'application autour de deux chemins bien distincts, en m'inspirant directement de l'architecture d'Uber. J'ai créé des routes d'API différentes pour l'OLTP et pour l'OLAP.

Une route API est une adresse web spécifique sur votre serveur (comme un "guichet" d'entrée) qui permet à votre interface visuelle ou à un autre programme de venir récupérer des données ou d'envoyer des commandes pour qu'elles soient traitées par votre code backend.

L'OLTP, pour Online Transaction Processing, c'est le chemin opérationnel. Ce sont les routes API utilisées par le jeu en direct. Elles ne font que deux choses très simples et très rapides : lire le prix en RAM, ou déposer un ordre dans la file d'attente. C'est ce qui garantit la fluidité du jeu.

L'OLAP, pour Online Analytical Processing, c'est le chemin analytique. J'ai créé une route API spécifique qui ne sert pas à jouer, mais à analyser les tendances, calculer les volumes d'argent totaux échangés ou générer des statistiques complexes. Cette route fait des calculs lourds, mais elle est complètement isolée du moteur de jeu. Si l'analyse prend du temps, le joueur ne le ressentira jamais.

Il nous reste alors un dernier risque de taille. Si les prix et les calculs sont faits dans la RAM du serveur, que se passe-t-il si le serveur s'éteint brutalement ou plante ? Toute la mémoire vive est effacée, et tous les joueurs perdent leur argent.

Pour sécuriser la donnée, j'ai implémenté le concept d'Event Sourcing. L'erreur classique serait de sauvegarder uniquement l'état final, par exemple "Le joueur A a 500 dollars". L'Event Sourcing fait l'inverse. Au lieu de stocker l'état final, on stocke chaque événement individuel qui s'est produit depuis la création du jeu de manière immuable.

Dans mon architecture, à chaque fois qu'une transaction est validée par le moteur, un événement "Le bot 3 a acheté 2 bois pour 15 dollars" est inscrit de manière persistante dans une base de données de sauvegarde. C'est un journal de bord qui garde la trace de tout. Si le serveur plante, il me suffit de relire ce journal depuis le début, de rejouer les événements un par un dans l'ordre, et je peux reconstruire instantanément l'état exact du marché et le portefeuille de chaque joueur. C'est d'ailleurs ce même principe de journal d'événements que Netflix ou Airbnb utilisent pour garantir l'intégrité de leurs données et reconstruire l'historique de leurs utilisateurs.

Ainsi, en combinant l'asynchrone, la séparation des lectures en RAM et des écritures, et l'Event Sourcing pour la sécurité, j'ai pu contourner les limites des bases de données traditionnelles pour créer un moteur fluide et robuste.

Partie Code :

Nous voici maintenant dans la troisième partie, où nous allons plonger sous le capot pour voir comment ces concepts se traduisent concrètement dans le code. Mon projet est divisé en deux fichiers distincts : le Backend, qui est le cerveau de l'application, et le Frontend, qui est l'interface visuelle.

Commençons par le cerveau. Pour sécuriser les transactions, j'ai implémenté le concept d'Event Sourcing. Au lieu d'écraser bêtement le solde d'un joueur dans la base de données, mon code utilise une base SQLite pour insérer une nouvelle ligne à chaque transaction valide. Il enregistre l'heure exacte, l'acteur, l'action, la ressource, la quantité et le prix. C'est un registre immuable. Grâce à cela, je ne perds aucune donnée historique et je pourrais, si besoin, reconstruire l'état financier exact de mon marché depuis son lancement.

Mais le véritable cœur de mon moteur, c'est la file d'attente asynchrone, déclarée simplement avec `order_queue = asyncio.Queue()`. C'est l'équivalent numérique d'une salle d'attente. C'est elle qui applique le motif "Fan-In", où de multiples producteurs envoient leurs données vers un point de rassemblement unique. Plutôt que de traiter chaque achat immédiatement et de risquer de bloquer le serveur, toutes les requêtes tombent instantanément dans cette file. En parallèle, une fonction que j'ai appelé le "moteur du marché" vient piocher dans cette file, ordre par ordre, pour les traiter à son propre rythme.

Pour alimenter cette file d'attente et simuler l'activité frénétique d'un MMORPG, j'ai écrit une fonction nommée `trading_bot`. Cette fonction tourne en boucle infinie en arrière-plan. Son rôle est simple : elle choisit aléatoirement une ressource, comme le bois ou l'or, choisit d'acheter ou de vendre, et génère une quantité au hasard. Ensuite, elle envoie cet ordre dans la file d'attente et se met en pause pendant une à trois secondes avant de recommencer. Comme cette fonction est asynchrone, je peux faire tourner des dizaines de bots en parallèle sans jamais monopoliser la puissance de mon ordinateur.

Pour pousser mon système dans ses retranchements, j'ai créé une route API spécifique : le "Tweet d'Elon Musk". C'est un "endpoint", c'est-à-dire une adresse web que l'on peut appeler depuis l'interface. Quand on clique sur le bouton "Tweet Positif" ou "Tweet Négatif", cette fonction se déclenche et utilise une simple boucle pour injecter instantanément 100 ordres massifs dans la file d'attente. L'API répond tout de suite "Ordres initiés", rendant la main à l'utilisateur, pendant que le serveur absorbe cette charge phénoménale en arrière-plan, sans sourciller.

Passons maintenant à l'interface visuelle, le Frontend. Celui-ci a été codé avec la bibliothèque Streamlit. La règle d'or ici, c'est que l'interface est complètement aveugle : elle ne fait aucun calcul métier.

Pour communiquer avec le cerveau, j'ai écrit une petite fonction clé : `fetch_data(endpoint)`. Son rôle est de faire le messenger. Elle va frapper à la porte de l'API du Backend, par exemple à l'adresse des prix, récupère les données brutes, et les traduit pour que l'interface puisse dessiner ses graphiques. J'y ai ajouté un bloc `try/except` de sécurité : si le serveur principal plante ou est éteint, la fonction ne renvoie juste « Rien » plutôt que de faire crasher violemment l'écran de l'utilisateur avec des lignes d'erreur incompréhensibles.

Enfin, pour que ce tableau de bord donne l'illusion du temps réel, j'utilise une astuce très simple à la toute fin de mon code Frontend, avec deux lignes : `time.sleep(1)` et `st.rerun()`. Normalement, une page web s'affiche une fois et s'arrête. Ici, je dis à mon code de mettre la page en pause pendant exactement une seconde, puis de se relancer entièrement depuis le début. Cela crée une boucle de rafraîchissement infinie. Chaque seconde, l'interface va chercher les nouveaux prix et met à jour les graphiques, offrant une expérience fluide et dynamique au joueur.

Partie Finale (Exemple + Conclusion) :

Pour conclure cette présentation, j'aimerais revenir sur ce qui a rendu ce projet particulièrement intéressant à réaliser. Ce qui m'a le plus passionné, c'est de réussir à faire le pont entre une idée purement "business" et ludique, ce simulateur de marché de jeu vidéo, et des concepts d'architecture et de code Big Data qui peuvent parfois sembler très abstraits sur le papier. Voir concrètement qu'une simple file d'attente asynchrone permet de sauver un serveur du crash lors d'une tempête d'écritures, c'est extrêmement formateur. Cela m'a vraiment permis de comprendre par la pratique l'importance vitale de séparer l'opérationnel de l'analytique et de découpler les lectures des écritures.

Bien sûr, ce simulateur n'est qu'un prototype fonctionnel et il reste de nombreuses pistes d'amélioration pour le rapprocher encore plus de la réalité industrielle.

Tout d'abord, au niveau de l'intelligence artificielle. Actuellement, les bots qui font vivre le marché achètent et vendent au hasard. Il serait très intéressant d'intégrer des modèles de Machine Learning, à l'image de ce que font des plateformes comme Airbnb ou Facebook pour personnaliser l'expérience. On pourrait entraîner ces bots à analyser les tendances du marché pour qu'ils réagissent de manière stratégique, par exemple en créant eux-mêmes des mouvements de panique ou des bulles spéculatives.

Ensuite, au niveau de l'infrastructure pure. Pour le moment, l'application tourne localement et la persistance est gérée par une base de données SQLite. Pour passer véritablement à l'échelle du Big Data et accueillir des milliers de vrais joueurs, la prochaine étape logique serait de déployer cette architecture dans le Cloud. Il faudrait remplacer notre file d'attente Python native par un outil industriel robuste de traitement d'événements comme Apache Kafka, que l'on retrouve chez Uber ou Facebook. Et enfin, il faudrait migrer notre stockage vers une véritable base de données NoSQL distribuée, comme Cassandra ou MongoDB pour garantir une haute disponibilité.

Je vous remercie pour votre attention tout au long de cette présentation, et je suis maintenant à votre entière disposition si vous avez des questions sur le code, l'architecture ou mes choix techniques.